

Optimization: Join

Knowledge objectives

1. Write the pseudo-code of two basic join algorithms
2. Explain the prerequisites of each basic join algorithm (i.e. Row Nested Loops, Block Nested Loops)
3. Explain the star-join pattern and how it is typically handled in relational DBMSs
4. Explain the meaning and benefits of pipelining joins
5. Enumerate three execution alternatives to star-joins
6. Explain the consequences of enabling star transformation in Oracle
7. Explain what a join-index is
8. Justify the usefulness of a join-index

Understanding Objectives

1. Use either Row Nested Loops, Block Nested Loops to join two tables
2. Justify the choice of a join algorithm in front of others given a multidimensional setting

Application Objectives

1. Given a decisional workload, a set of dimensional tables including tuples and a constraint in terms of space, define the best structures for these tables that fit in the space and optimize the cost provided by Oracle's query optimizer

Row Nested Loops
Block Nested Loops

JOIN

Row Nested Loops

Algorithm

```

for each block of R
  read block of R into a memory page
  for each tuple t in the page read
    if there is an index in S for attribute A
      go through the index of S.A using the value of t.A
      if there is any tuple satisfying the join condition
        if we are interested in attributes of S
          go to the corresponding tuples of S
        endIf
      generate result
    endIf
  else scan the whole table S and generate result
  endIf
endForEach
endForEach

```

Cost, if we do NOT look for attributes of S (semi-join)

■ B+: $B_R D + |R| \cdot (h_S \cdot D + ((k-1)/u_S) \cdot D)$

Cost, if we DO look for attributes of S

■ B+: $B_R D + |R| \cdot (h_S \cdot D + ((k-1)/u_S) \cdot D + k \cdot D)$

■ Clustered: $B_R D + |R| \cdot (h_S \cdot D + D + (1.5(k-1)/R_S) \cdot D)$

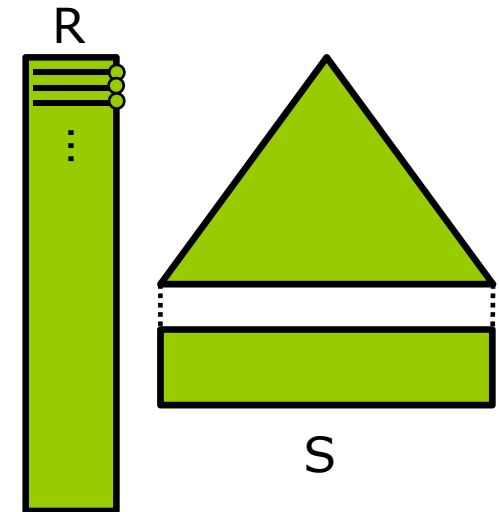
Considerations

- It is only useful if there is an index over the join attribute
- We cannot use this algorithm if we already performed another operation over the table (we could decide to change the syntactic tree)

$$u = \%load \cdot 2d = (2/3) \cdot 2d$$

$$h = \lceil \log_u |T| \rceil - 1$$

k = average appearance of each value of R.A in S.A



Row Nested Loops

Algorithm

```

for each block of R
  read block of R into a memory page
  for each tuple t in the page read
    if there is an index in S for attribute A
      go through the index of S.A using the value of t.A
      if there is any tuple satisfying the join condition
        if we are interested in attributes of S
          go to the corresponding tuples of S
        endIf
      generate result
    endIf
  else scan the whole table S and generate result
  endIf
endForEach
endForEach

```

Cost, if we do NOT look for attributes of S (semi-join)

■ B+: $B_R D + |R| \cdot (h_S \cdot D + ((k-1)/u_S) \cdot D)$

Cost, if we DO look for attributes of S

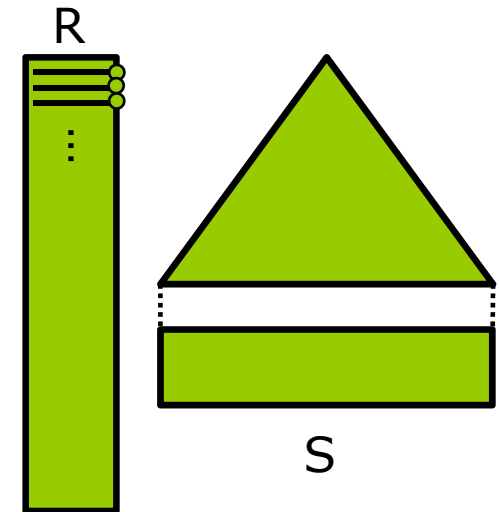
■ B+: $B_R D + |R| \cdot (h_S \cdot D + ((k-1)/u_S) \cdot D + k \cdot D)$

■ Clustered: $B_R D + |R| \cdot (h_S \cdot D + D + (1.5(k-1)/R_S) \cdot D)$

Considerations

- It is only useful if there is an index over the join attribute
- We cannot use this algorithm if we already performed another operation over the table (we could decide to change the syntactic tree)

$u = \%load \cdot 2d = (2/3) \cdot 2d$
 $h = \lceil \log_u |T| \rceil - 1$
 $k = \text{average appearance of each value of R.A in S.A}$



Row Nested Loops

Algorithm

```

for each block of R
  read block of R into a memory page
  for each tuple t in the page read
    if there is an index in S for attribute A
      go through the index of S.A using the value of t.A
      if there is any tuple satisfying the join condition
        if we are interested in attributes of S
          go to the corresponding tuples of S
        endIf
      generate result
    endIf
  else scan the whole table S and generate result
  endIf
endForEach
endForEach

```

Cost, if we do NOT look for attributes of S (semi-join)

■ B+: $B_R D + |R| \cdot (h_S \cdot D + ((k-1)/u_S) \cdot D)$

Cost, if we DO look for attributes of S

■ B+: $B_R D + |R| \cdot (h_S \cdot D + ((k-1)/u_S) \cdot D + k \cdot D)$

■ Clustered: $B_R D + |R| \cdot (h_S \cdot D + D + (1.5(k-1)/R_S) \cdot D)$

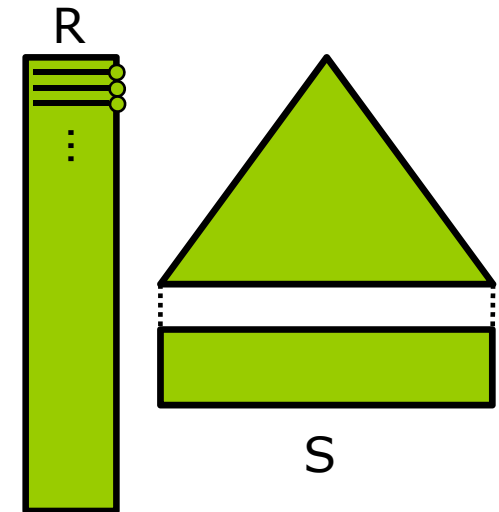
Considerations

- It is only useful if there is an index over the join attribute
- We cannot use this algorithm if we already performed another operation over the table (we could decide to change the syntactic tree)

$$u = \%load \cdot 2d = (2/3) \cdot 2d$$

$$h = \lceil \log_u |T| \rceil - 1$$

k = average appearance of each value of R.A in S.A



Block Nested Loops (I)

□ Algorithm

```

for each block of R
  read block of R into a memory page
  for each block of S
    read block of S into a memory page
    for each tuple t in the page of R
      for each tuple s in the page of S
        if (t.A  $\theta$  s.B) then generate result
      endIf
    endForEach
  endForEach
endForEach

```

□ Cost (with $M+2$ memory pages)

- $B_R \cdot D + B_S \cdot \lceil B_R/M \rceil \cdot D$

□ Considerations

- It can always be used (even for θ -join)
- It is of special interest if $B_R \leq M$
- It is not symmetric
 - It is better if the smaller table is in the external loop (we must use the commutative property of join, if necessary)

Block Nested Loops (II)

M=2

Accesses

E	E	I	O
R ₃	R ₂	S ₁	

R₁ R₂ R₃ R₄
S₁ S₂

R
R
R
R
R
R
R
R
R

R
R
R
R
R
R

E	E	I	O
R ₁	R ₂	S ₃	

R₁ R₂
S₁ S₂ S₃ S₄

Star-join
Pipelining
Join-index

ADVANCED JOINS

Star-joins

- ❑ Pattern
 - One big (i.e., fact) table
 - Many smaller (i.e., dimension) tables
 - Multiple selection clauses over dimensions
 - No join between dimension tables
- ❑ Easy to make mistakes in cost estimation
 - Do not look for the optimum, but a “robust” plan
- ❑ Rewrite the query
 - a) To use materialized views
 - b) To generate sub-queries
- ❑ Apply heuristics
 - Use indexes to access the fact table
 - ❑ Specially bitmaps
 - Do not avoid cartesian product of dimensions
 - ❑ After filtering they might not be that big

Star-join in Oracle

```
ALTER SESSION SET
  star_transformation_enabled = TRUE;
```

■ Enables query rewriting

```
SELECT ...
FROM Sales f, Product d1, Place d2, Time d3
WHERE f.productId=d1.ID AND f.placeId=d2.ID AND h.timeId=d3.ID
      AND d1.articleName IN ('Ballpoint','Rubber')
      AND d2.region='Catalunya'
      AND d3.month IN ('January02','February02')
GROUP BY ...
```

For one or all dimensions



```
SELECT ...
FROM Sales f
WHERE f.productId IN (SELECT d1.ID FROM Product d1 WHERE d1.articleName IN ('Ballpoint','Rubber')) )
      AND f.placeId IN (SELECT d2.ID FROM Place d2 WHERE d2.region='Catalunya')
      AND h.timeId IN (SELECT d3.ID FROM Time d3 WHERE d3.month IN ('January02','February02'))
GROUP BY ...
```

■ Bitmaps must exist

- They can be generated on the fly from B-trees

■ Joining back the dimension tables can be avoided

- Keep data in memory

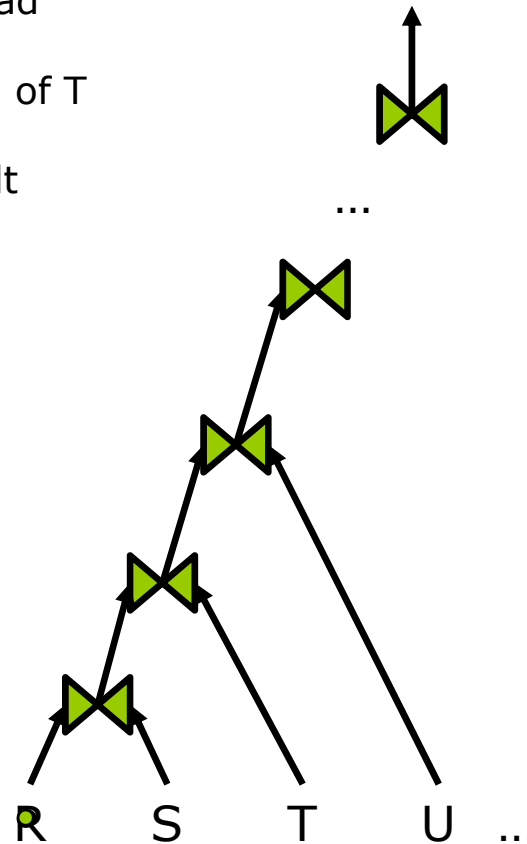
Pipelining

□ Algorithm

```

for each block of R
  read block of R into a memory page
  for each tuple t in the page read
    if matches any tuple of S
      if matches any tuple of T
        ...
        generate result
      ...
    endIf
  endIf
endForEach
endForEach

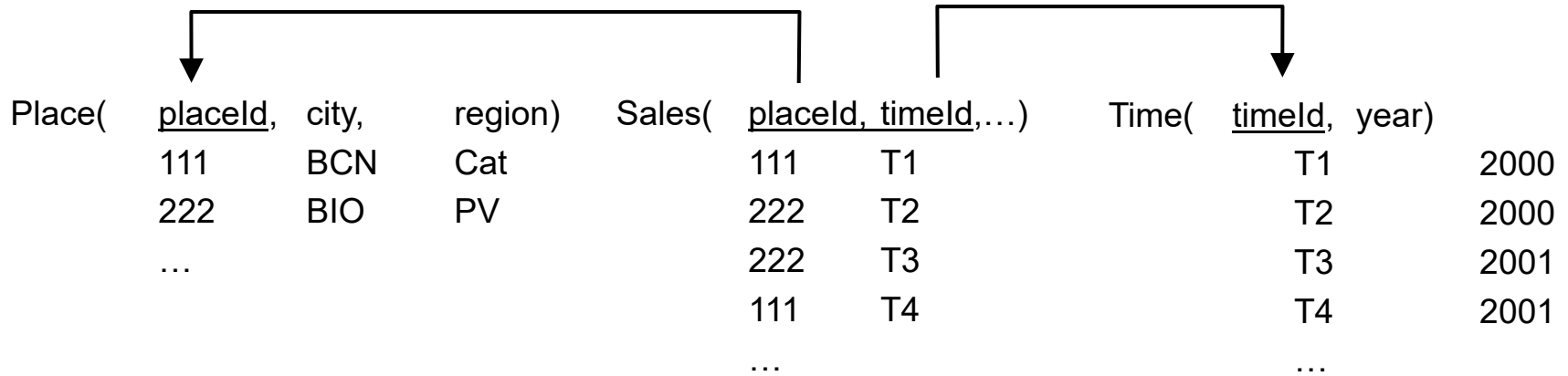
```



Pipelining

- ❑ Usually, the DBMSs try to avoid materializing intermediate results
- ❑ If a join is calculated by means of Nested Loops (either Block or Row), the table in the external loop is not materialized
 - If we have a left-deep tree, we will not materialize anything
- ❑ If nothing is materialized, access paths will tell the cost of the query

Join-index (I)



Join-index (one attribute)

Cat	1,4, ...
PV	2,3, ...

Join-index (two attributes)

Cat	2000	1, ...
	2001	4, ...
PV	2000	2, ...
	2001	3, ...

Join-index (two attributes)

2000	Cat	1, ...
	PV	2, ...
2001	Cat	4, ...
	PV	3, ...

Benefits of Bitmap-join-index

```
CREATE BITMAP INDEX salesRegion  
ON Sales(Place.region)  
FROM Sales, Place  
WHERE Sales.placeId=Place.ID;
```

- ❑ The algorithm would be similar to Row Nested Loops
- ❑ The saving in multidimensional queries is that dimension tables do not need to be accessed

Join-index (II)

□ Algorithm

go through the index of R.A using the values in the select clause
if there is any tuple satisfying the join condition
 if we are interested in attributes of S
 go to the corresponding tuples of S
 endIf
 generate result
endIf

$$u = \%load \cdot 2d = (2/3) \cdot 2d$$

$$h = \lceil \log_u(BR) \rceil - 1$$

k = average appearance of
each value of R.A in S.A

v = constants in the select clause

□ Cost, if we do NOT look for attributes of S (semi-join)

■ B+: $v \cdot (h_S \cdot D + ((k-1)/u_S) \cdot D)$

■ Bitmap: $v \cdot \lceil |S|/\text{bits} \rceil \cdot D$

□ Cost, if we DO look for attributes of S

■ B+: $v \cdot (h_S \cdot D + ((k-1)/u_S) \cdot D + k \cdot D)$

■ Clustered: $v \cdot (h_S \cdot D + D + (1.5(k-1)/R_S) \cdot D)$

■ Bitmap: $v \cdot \lceil |S|/\text{bits} \rceil \cdot D + (B_S \cdot (1 - (1 - SF)^{R_S})) \cdot D$

□ Considerations

- It is really useful if there is a join-index over the selection attribute of R
- It may be useful even if there is a join-index over the join attribute of R
- We cannot use this algorithm if we already performed another operation over the table (we could decide to change the syntactic tree)

CLOSING

Summary

- ❑ Basic join algorithms
 - Row Nested Loops
 - Block Nested Loops
- ❑ Multidimensional queries
 - Star-join
- ❑ Pipelining
- ❑ Join-index
- ❑ Star-join algorithms

Bibliography

- ❑ M. Golfarelli and S. Rizzi. *Data Warehouse Design*. McGraw-Hill, 2009
- ❑ C. S. Jensen, T. B. Pedersen, C. Thomsen. *Multidimensional Databases and Data Warehousing*. Morgan&Claypool Publishers, 2010
- ❑ Oracle Database. *Data Warehousing Guide*. 11g Release 2, September 2011